

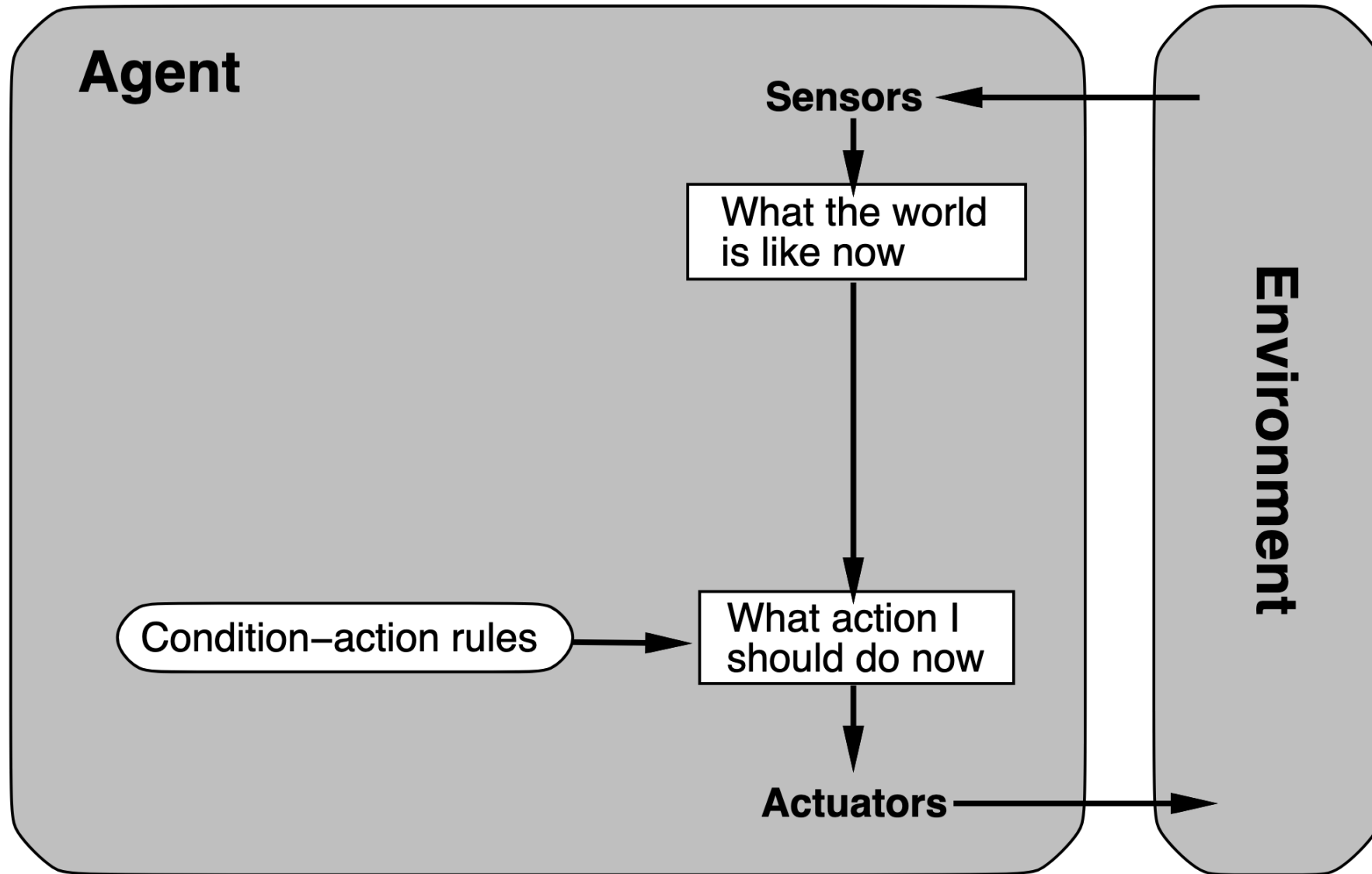
LECTURE 3

AGENT REASONING & LEARNING

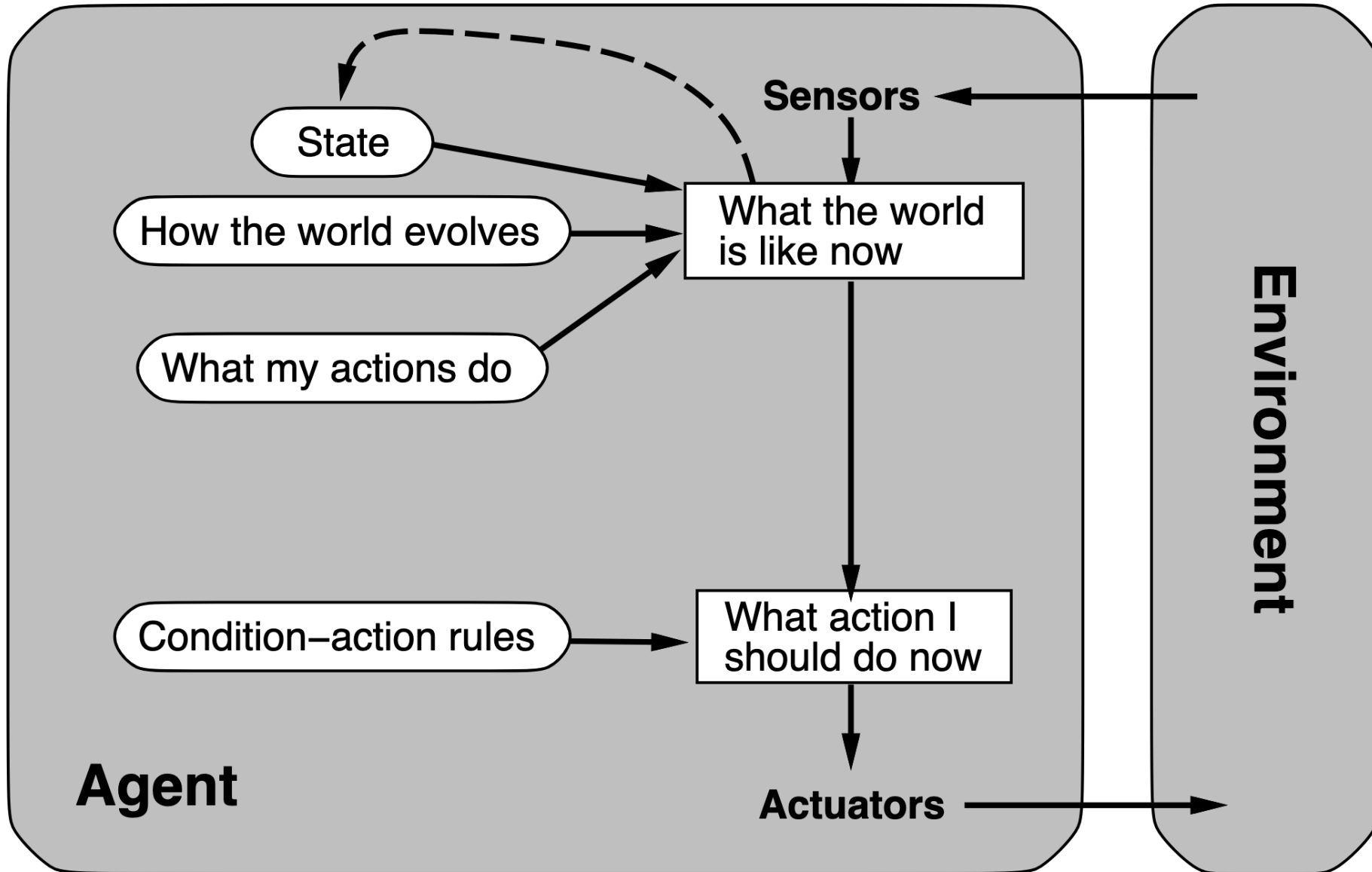
Mirgita Frasheri | mirgita.frasheri@ece.au.dk

Lukas Esterle | lukas.esterle@ece.au.dk

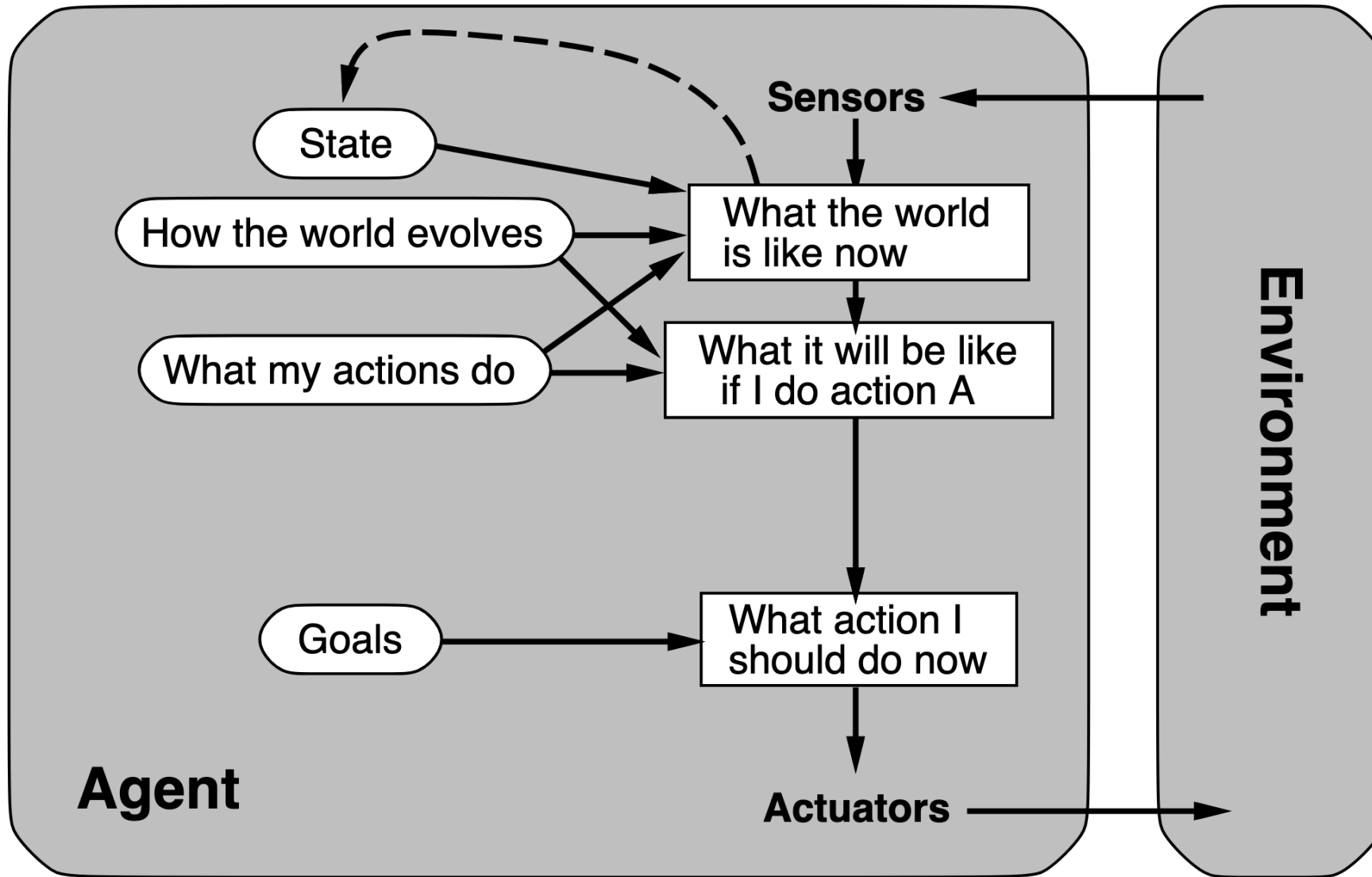
REFLEX AGENTS



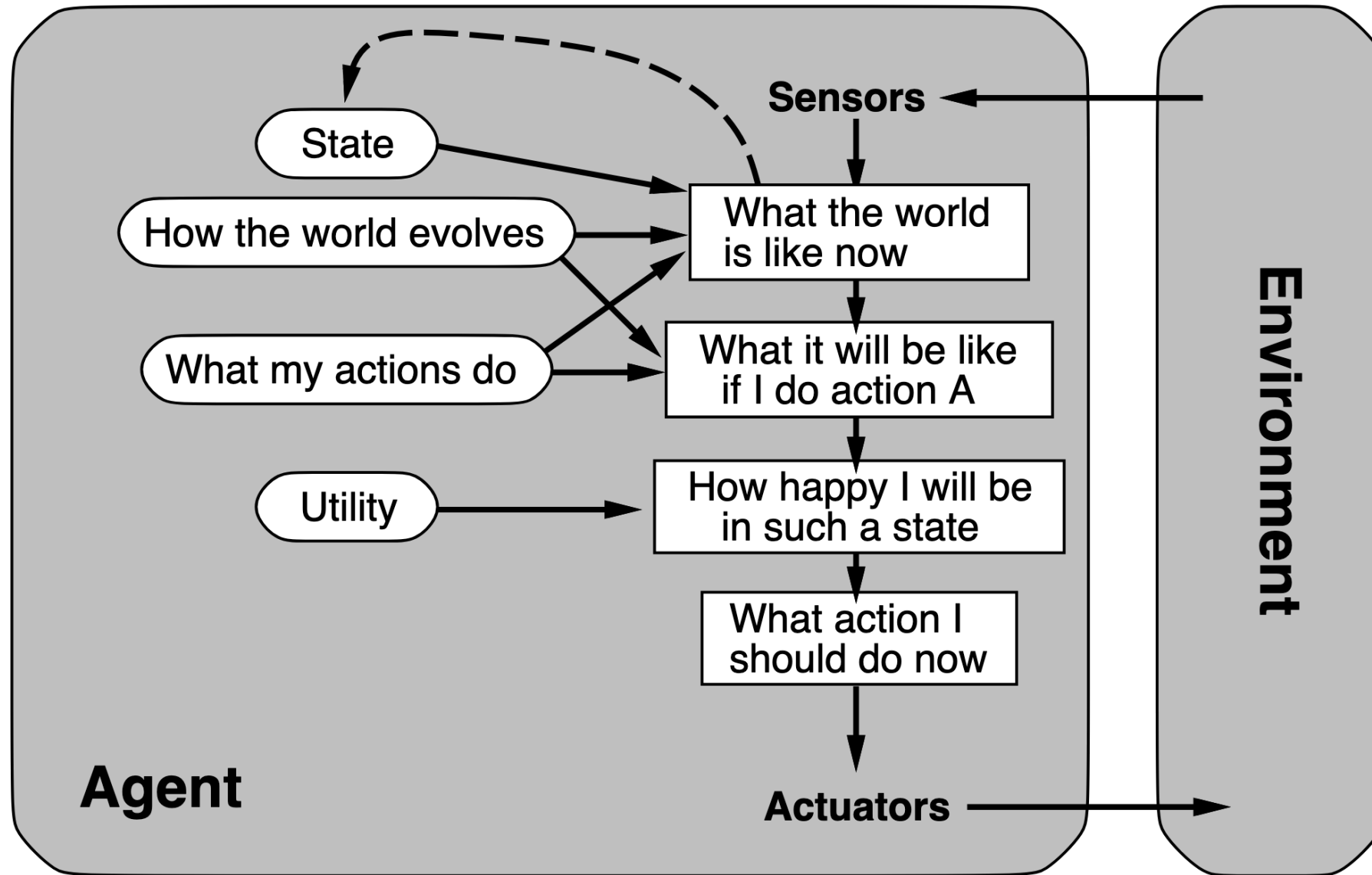
MODEL-BASED AGENTS



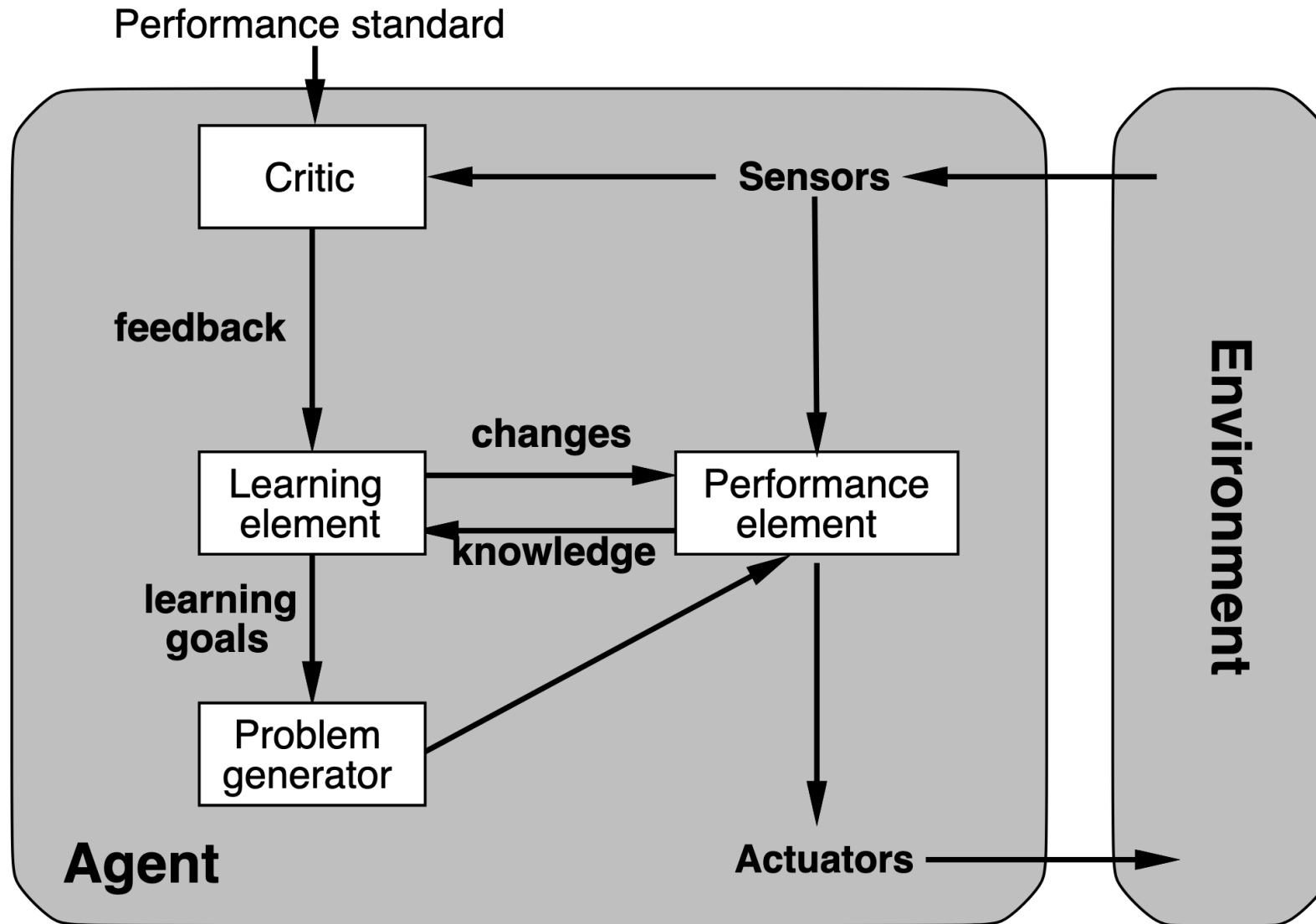
GOAL-BASED AGENTS



UTILITY-BASED AGENTS



LEARNING AGENTS



QUICK DIVE INTO MDPS

MARKOV DECISION PROCESSES (MDPs)

A sequential decision problem for a fully observable, stochastic environment with a Markovian transition function and additive reward is called a Markov Decision Process, and defined by a tuple $\langle S, A, T, R \rangle$

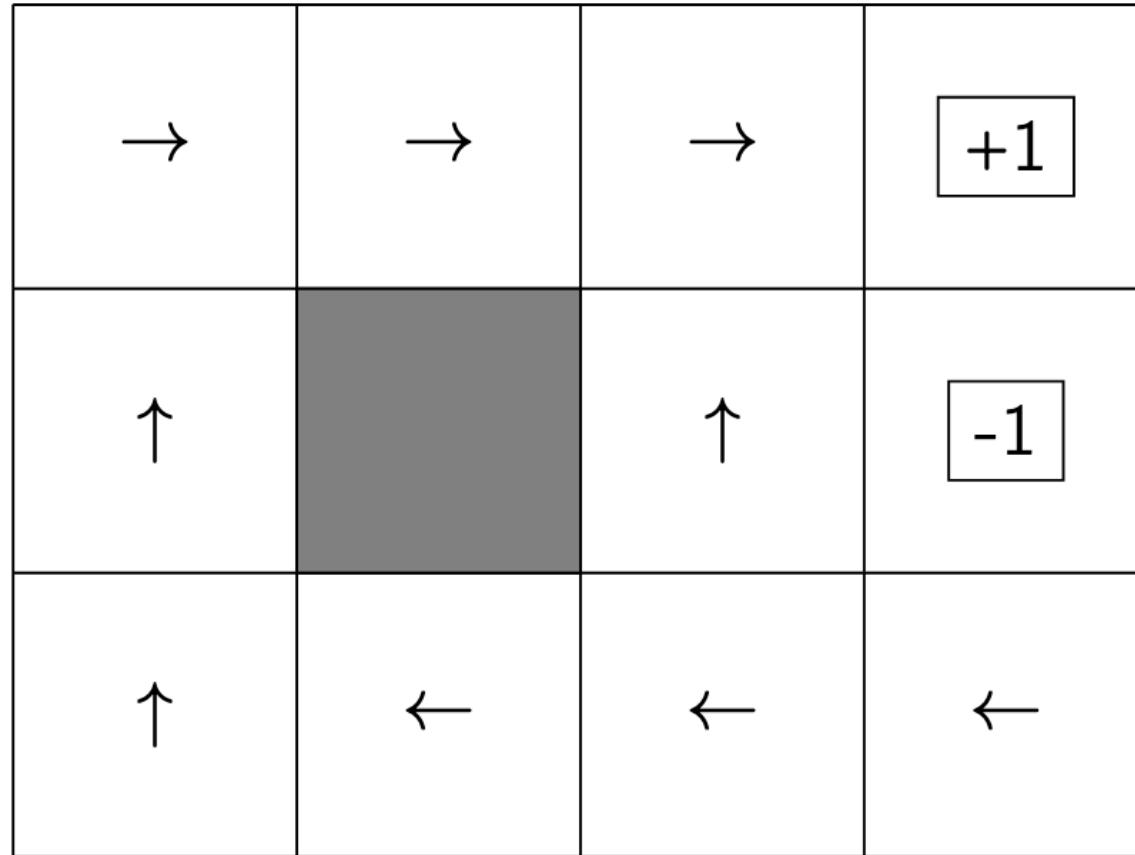
- A set of states $s \in S$
- A set of actions $a \in A$
- A stochastic transition function $T(s, a, s') = P(s' | s, a)$
- A reward function $R(s)$

-0.04	-0.04	-0.04	+1
-0.04		-0.04	-1
START	-0.04	-0.04	-0.04

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

-0.04	-0.04	-0.04	+1
-0.04		-0.04	-1
START	-0.04	-0.04	-0.04

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state



A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy?

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

$$V^\pi(s) = R(s) + \gamma \sum P_{s\pi(s)}(s') V^\pi(s')$$

Bellman's equation

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P_{s\pi(s)}(s') V^\pi(s')$$

Immediate reward

Future reward

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P_{s\pi(s)}(s') V^\pi(s')$$

System of linear equations

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

$V^*(s)$, the optimal value function

$$V^*(s) = \max_{\pi} V^\pi(s)$$

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

$V^*(s)$, the optimal value function

$$V^*(s) = R(s) + \max_a \sum P_{s\pi(s)}(s') V^*(s')$$

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

$V^\pi(s)$, the value function for π :
expected total payoff for starting in s and executing π

$V^*(s)$, the optimal value function

π^* , the optimal policy

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} \sum P_{s\pi(s)}(s') V^*(s')$$

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

How to find $V^*(s)$?

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

How to find $V^*(s)$?

Value iteration:

Repeatedly apply Bellman's equation (find V^*)

At the end derive π^* (argmax)

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

How to find $V^*(s)$?

Value iteration:

Repeatedly apply Bellman's equation (find V^*)

At the end derive π^* (argmax)

Policy iteration:

Compute a new policy every iteration

- Evaluate policy (linear system)
- Improve policy (with argmax)

A policy is a function $\pi : S \rightarrow A$ that specifies what action the agent should take in any given state

Quality of the policy? **Utility as a function of rewards**

How to find $V^*(s)$?

Value iteration:

Repeatedly apply Bellman's equation

At the end derive π^*

Policy iteration:

Compute a new policy every iteration



Good for
small
state
space

Some big assumptions: we know T and R (model of the environment)

Some big assumptions: we know T and R

Reinforcement learning

Some big assumptions: we know T and R

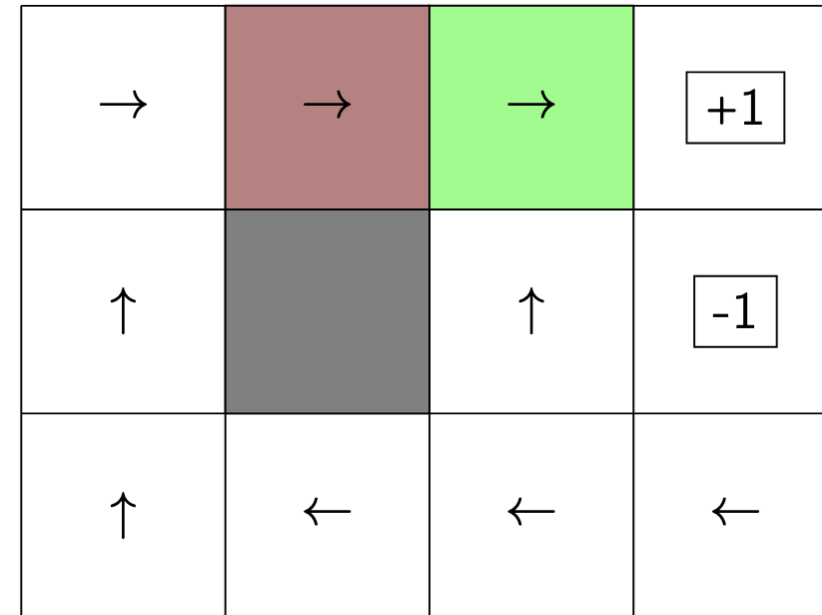
Reinforcement learning



- In passive RL, the agents' policy π is fixed, it only needs to know how good it is
- Direct utility estimation method
 - Agent runs a number of trials, starting in START and continuing until it reaches a terminal state
- The utility of a state is the expected total remaining reward (reward-to-go)
- Each trial provides a sample of the reward-to-go for each visited state
- The agent keeps a running average for each state, which will converge to the true value

Trials

(1,1) $\uparrow$ -0.04	(1,1) -0.04
(1,2) $\uparrow$ -0.04	(1,2) $\uparrow$ -0.04
(1,3) $\rightarrow$ -0.04	(1,3) $\rightarrow$ -0.04
(2,3) $\rightarrow$ -0.04	(2,3) $\rightarrow$ -0.04
(3,3) $\rightarrow$ -0.04	(3,3) $\rightarrow$ -0.04
(3,2) $\uparrow$ -0.04	(3,2) $\uparrow$ -0.04
(3,3) $\rightarrow$ -0.04	(4,2) exit -1
(4,3) exit +1	(done)
(done)	



$$\gamma = 1, R = -0.04$$

$$V(2, 3) \approx (0.840 - 1.12)/2 = -0.14$$

$$V(3, 3) \approx (0.96 + 0.88 - 1.08)/3 = 0.86$$

- In passive RL, the agents' policy π is fixed, it only records how good it is
- Direct utility estimation method
 - Agent runs a number of trials, starting from a fixed state and reaches a terminal state
 - The utility of a state is the expected return (reward-to-go)
 - Each trial provides a sample of the reward-to-go for each visited state
 - The agent keeps a running average for each state, which will converge to the true value

State Utilities are dependent (as in the Bellman equation)

- In passive RL, the agents' policy π is fixed, it only needs to know how good it is
- Adaptive Dynamic Programming
 - An ADP agent uses dependencies between state utilities to speed up value estimation
 - It follows a policy π and can use observed transitions to incrementally build the transition model $P(s'|s, \pi(s))$
 - It can then plug the learned transition model and observed rewards $R(s)$ into the Bellman equations to $V(s)$
 - The equations are linear because there is no max operator $\rightarrow$ easier to solve
 - The result is $V(s)$ for the given policy π

- In passive RL, the agents' policy π is fixed, it only needs to know how good it is
- Temporal Difference (model-free)
 - TD is another passive utility value learning algorithm
 - TD uses a learning rate parameter α to select the rate of change of utility adjustment
 - TD does not need a transition model to perform its updates, only the observed transitions

$$V^{\pi}(s) \leftarrow V^{\pi}(s) + \alpha(R + \gamma V^{\pi}(s') - V^{\pi}(s))$$

- In passive RL, the agents' policy π is fixed, it only needs to know how good it is
- Temporal Difference (model-free)
 - TD is another passive utility value learning algorithm
 - TD uses a learning rate parameter α to select the rate of change of utility adjustment
 - TD does not need a transition model to perform its updates, only the observed transitions

$$V^{\pi}(s) \leftarrow V^{\pi}(s) + \alpha(R + \gamma V^{\pi}(s') - V^{\pi}(s))$$

- In passive RL, the agents' policy π is fixed
- Temporal Difference (model-free)
- TD is another passive utility value function
- TD uses a learning rate parameter α for utility adjustment
- TD does not need a transition model, but it is limited to the observed transitions

While ADP adjusts the utility of s with all its successor states, TD learning adjusts it with that of a single successor state s' .

- While a passive RL agent executes a fixed policy π , an active RL agent has to decide which actions to do
- The active RL agent may select maximizing actions based on a faulty learned model, and fail to incorporate observations that might lead to a more correct model
- To avoid this, the agent design could include selecting actions that lead to more correct models at the cost of reduced immediate rewards
- Exploitation vs. exploration trade-off

- Q-learning (TD learning + exploration)
- Q-value (Action-Value), utility for performing a in s

- Q-learning (TD learning + exploration)
- Q-value (Action-Value), for each state

$$Q^{\pi}(s,a) \leftarrow Q^{\pi}(s,a) + \alpha(R + \gamma \max_{a'} Q^{\pi}(s',a') - Q^{\pi}(s,a))$$

- Q-learning (TD learning + exploration)
- Q-value (Action-Value), for each state

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$;
 until S is terminal

- Another variant to this, SARSA

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Repeat (for each step of episode):

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

15 MINUTES QUESTIONS

Questions group 1:

What are the main paradigms for building agent architectures?

How are agents defined and what are their main characteristics?

Questions group 2:

Pick any two architectures and compare them. What are the strengths of each?

What are other concepts similar to agents?

Questions group 3:

Compare the MAPE-K and the self-aware architectures.

How do agents differ from similar/related concepts?

Questions group 4:

Reflect on the architectures you have learned about, and suitable applications where they could be used.

What are some challenges/concerns wrt deployment of agents/robots in the real-world?

RL EXERCISE

- Teach the the robot to drive in a circle
 - *Hint: use distance to an obstacle to give reward
- Teach it to drive without hitting obstacles